BNN FCC Design Contest

Miguel Sanchez, Alejandro Paez-Sansonetti

Reconfigurable Computing 2, University of Florida

I. Summary

Targeted tradeoff

Maximizing throughput with no cap on latency

Design Performance and Area:

- ***Max Frequency:*** *945.179 MHz*
- ***LUT:*** *15,568*
- ***FF:*** *34,328*
- ***LUTAsMem:*** *713*
- ***BRAM:*** *53*

Passing Testbench: *These are measured with the bnn_fcc_verify.yml file in repository with CONFIG_VALID_PROBABILITY=1; DATA_IN_VALID_PROBABILITY = 1*

- ***Throughput:*** *112.5 cycles/output*
- ***Latency:*** *394.4 cycles*

```
(base) (openflex) [apaezsansonetti@ece-lnx-10 openflex]$ openflex bnn_fcc_verify.yml
#
# Stats:
# Avg latency (cycles) per image: 395.4 cycles
# Avg latency (time) per image: 3954.2 ns
# Avg throughput (outputs/sec): 888730.9
# Avg throughput (cycles/output): 112.5
#  quit -f
# End time: 17:30:34 on Apr 21,2026, Elapsed time: 0:00:04
# Errors: 0, Warnings: 0
# *** Summary ********************************************
#     qrun: Errors:   0, Warnings:   0
#     vlog: Errors:   0, Warnings:   0
#     vopt: Errors:   0, Warnings:  48
#     vsim: Errors:   0, Warnings:   0
#   Totals: Errors:   0, Warnings:  48
(base) (openflex) [apaezsansonetti@ece-lnx-10 openflex]$
```

Parameters for Provided Testbench: We added the parameter **Layer parallel inputs** to control PW seen by each hidden layer. The Parallel neurons parameter was already in the given testbench and dictates the amount of neuron processors that work in parallel for a given hidden layer. Other configurations work but we found this topology produced the best results for our design. In order for the testbench to simulate our implemented design the following parameters must be set to as followed:

- LAYER_PARRALEL_INPUTS: {64,32,32}
- PARRALEL_NUERONS: {32,32,10}

Overview: *We designed and verified a structurally based BNN accelerator that achieves 945.179 MHz, starting from an initial implementation at 91.4 MHz and improving performance through over 300 iterative optimizations. This was accomplished by analyzing hundreds of post-route, post-place, and synthesis reports, along with extensive fanout and congestion analysis.*

There is only one FSM in our entire design, consisting of four states. This FSM is located in our configuration manager, which is instantiated only once. Other than that, everything else is implemented using datapath control signals, pipelined control signals, and careful phase alignment.

Phase alignment refers to how and when signals propagate through the datapath. This was crucial to our optimization process, as it allowed us to open up several avenues of simplification that were not possible before. This enabled us to significantly reduce area, fanout, and congestion while still passing both the provided testbench and the expanded testbench we developed.

We strongly believe that the remaining limitation preventing us from reaching 1 GHz is the final FSM in the system. Given more time, we would likely be able to remove it and achieve timing closure at 1 GHz.

II. Design

A. Neuron Processor

The neuron processor's top level is NP_UNIT.sv. NP_UNIT.sv comprises two smaller submodules, NP_DP and NP_FSM. Although it is named NP_FSM, there is no state machine or enum-based logic. Instead, we control NP_DP using structural control signals that are timed in accordance with the valid_in and last_in signals.

The neuron processor has an initial latency of 14 and uses pipelined control signals to guarantee a steady throughput of 1.

Each neuron processor that is not part of the output layer has its own weight RAM and threshold RAMs, which pass through a 10+ stage sink register pipeline before being delivered to the neuron processor at the correct time. Nothing in this weight and threshold pipeline requires a reset or clear, as no data is meaningful unless the valid_in pipeline is properly aligned in phase.

Neuron processors in the output stage omit the threshold RAMs and associated pipeline, as they are not needed and would otherwise result in wasted circuitry.

***Control**: The controller dictates when valid_out, valid_accumulate, acc_ld, and acc_en are asserted. Originally, we used a state machine to accomplish this. However, after reaching 800 MHz, this design exhibited excessive fanout, leading to significant congestion and frequent appearance in critical paths identified in post-route reports.*

To address this, we leveraged the deterministic nature of our system. We know that valid_out must be asserted LAT cycles after last_in arrives, and that valid_acc must be asserted at LAT − 1. We also know that acc_ld should be asserted 11 cycles after each valid_in, and that acc_en should be asserted LAT − 3 cycles after last_in.

Additionally, we used this deterministic behavior to reduce the number of resets, further decreasing fanout and congestion in already dense regions. Our neuron processor only uses an asynchronous reset on the valid_in pipeline that originates from NP_Unit.sv and passes through NP_FSM.sv. A single valid gate is placed in NP_Unit.sv before last_in reaches the controller, ensuring that only one valid gate is required for the entire neuron processor.

***Data Path:** Our neuron processor data path (NP_DP.sv) consists of 4 pipelined modules that each have their own latency. The flow is simple:*

- ***xnor -> popcount -> accumulate -> threshold.***

For the final hidden layer we omit the threshold side thus its r latency is 13 cycles. This is why we made a separate module despite the recommendation to just use the same design for all layers.. This immittance allows us to save resources and routing.

1. ***NP_XNOR_UNIT.sv**: this unit is fairly simple, it has one sink register for the incoming weights and data, then in the next cycle computes a registered xnor operation of these two inputs. In total this has a latency of two cycles*
2. ***NP_Pop.sv**: our popcount module does not compute the typical pop count of simply, count how many ones are in the array. Instead we make this operation a pipelined tree structure. This pipelined tree structure splits up the arrays, determines how many ones then sums them up over multiple cycles. There is also a sink register for the original inputs and sink registers in between stages that consume a lot of logic. In total this unit has a latency of 8 cycles.*
3. ***NP_Accum.sv**: The accumulation unit has a pipeline delay of 3. The first stage is just a regular sink stage to collect the data from the popcount unit. We then go to a sum stage where if enabled by the controller we add the popcount to the sum. We then move onto the select stage, if ld is pulsed high from the controller the final output of the accumulate is loaded in from the summation stage.*
4. ***NP_Threshold.sv:** The threshold unit is the only module with an initial latency of one. It simply computes if the accumulated sum is greater than the specified threshold from the RAM. It is important to note that there is a sink register for the threshold but because it is loaded before the sum arrives it does not contribute to latency.*

B. Memory Units and Wrappers

One of the biggest considerations we had when designing our systems was what type of memory to use and how to implement them. For example LUT based memory is usually faster than block memory but is worse for routing. This became very apparent at 600Mhz when almost every critical path was LUT memory related caused by large routing and not logic depth. Bram on the other hand is slower but routes a lot better, making it more predictable when optimizing.

***Testing memory units:** In order to determine which type of memory would achieve the highest Fmax, we created two wrappers: BRAM_LUT_SINK and BRAM_SINK. Both wrappers include a sink register for incoming data on the input path and instantiate a single-port memory. The LUT version instructs Vivado to use distributed memory, while BRAM uses block RAM.*

After several tests, we determined that the most routing-friendly (and Fmax-friendly) combination was to use block RAM for storing weights and LUT-based memory for everything else. In hindsight, this should have been obvious—the number of weights we need to store is far greater than any other data (such as thresholds). The large size of the weight memories results in wide address fields, making LUT-based multiplexing a routing bottleneck. After switching weight memory to block RAM and adding the sink register, our reported frequency increased past 700 MHz.

After this change, our next bottleneck was the block RAM itself. While it improved routing, it remained fundamentally slower. Regardless of the optimizations we applied, the output datapath (in this case, weights being delivered to the neuron processors) consistently appeared as the worst path. Even after adding a source register between the RAM output and the neuron processor weight pipeline, we only gained about 20 MHz.

After researching online forums, we discovered a specialized Vivado IP called xpm_memory. This IP allows the creation of a single-port memory with a configurable read latency. When the read latency is set to two, Vivado places a specialized sink register close to the memory output port. This register is physically closer than any manually inserted register. To verify this, we created another version of the LUT RAM, called BRAM_LUT_SINK2, which included one sink register on the input side and one on the output side to mimic the xpm behavior. The results were clear: the xpm IP does a

significantly better job at reducing routing length at the output.

The next memory-related bottleneck was on the input side. At around 780 MHz, even with a sink register on the input, the routing and logic depth were still too large. To address this, we set the xpm read enable port to 1, meaning it was always reading. This was possible because downstream logic is unaffected when the valid pipeline is out of phase with the RAM output pipeline.

NP_MEM.sv *is a top level module that houses our neuron processor and their respective weight and threshold memories. There are sink and source registers for incoming enables,valid and last flags, and larger buses. This module does nothing else but act as a pipeline for the signals that need to reach the neuron processor and their respective memories.*

Once again the only pipeline that is globally reset is the valid_in pipeline. This is also where the last_in in valid gates before reaching the neuron processor. . It is important to note that the weight rams are always read from as it doesn't matter what they output if the valid_in pipeline is not in phase.

C. Layer Design

Each layer is made of three main modules in our design: a configuration controller, activation input buffer, and compute layer. The configuration controller loads the weight and threshold memories for that layer. The activation input buffer stores the current vector and next vector in a double-buffer. The computer layer reads a bank of the current activation buffer, gets the weights and thresholds that match and drives a parallel bank of neurons.

Each layer controller generates addresses and walks all beats of a neuron then advances to the next neuron group (threshold addresses only worry about group_idx).

- *w_addr = group_idx * beats + beat_idx*

Here is our layer structure with PW being the parallel width, PN being the parallel neuron processors. Beat meaning how many cycles it takes for a neuron to be completed in a neuron processor (after initial latency. While groups is how many neurons each neuron processor is responsible for processing.

Layer	*PW*	*PN*	*beats*	*groups*
H0	*64*	*32*	*12.25*	*8*
H1	*32*	*32*	*8*	*8*
OUT	*32*	*10*	*8*	*1*

Layer Controller:

The purpose of the layer controller is to read the outputs of the previous layer from the input buffer and deliver them to the neuron processors. Additionally, it is responsible for sending valid_in, last_in, and the necessary weights and thresholds to the correct neuron processor at the appropriate time.

Originally, this functionality was implemented using a four-state FSM. However, once we exceeded 900 MHz, clock enable (CE) and reset (R) routing began to dominate every critical path. To address this, we gradually transitioned to a more structured design, eventually removing the FSM entirely.

Our final implementation relies on phase alignment. Data being sent through the datapath must be aligned with the valid pipeline; otherwise, it does not affect downstream behavior

D. Customizable fanout tree pipeline

After reaching 700 MHz, it became clear from fanout and congestion reports that fanout was our primary Fmax bottleneck. To address this, we needed a way to create a registered fanout tree capable of supporting both single-bit and multi-bit bus transfers. Additionally, this fanout tree needed to be configurable in terms of depth and clustering while still ensuring that the maximum fanout seen by its intermediate registers is at max two. Thus ensuring that the final fanout seen downstream is never less than the fanout seen in the fanout tree itself.

Depth refers to how many cycles it takes for a signal to propagate through the tree, while clusters refer to how many registered copies are produced. For example, if a single-bit valid signal is driving 32 neuron processors and the number of clusters is set to 32, the pipeline of registered copies would be: 1 → 2 → 4 → 8 → 16 → 32. Thus, the fanout pipeline depth is $\log_2(\text{clusters}) + 1$.

While it may seem beneficial for each neuron processor to have its own dedicated copy of a signal, this comes with tradeoffs. Increased depth leads to exponential growth in the number of registers synthesized, which increases both area usage and routing congestion.

By parameterizing the number of clusters, we were able to experiment with different configurations to determine the optimal balance between routing efficiency, area, and Fmax. This fanout tree module effectively acted as an "optimization knob." Similar to how verification engineers use parameters to test functionality, we used this parameter to explore optimization tradeoffs.

In addition, we created two versions of the fanout tree pipeline: one with a global reset and one without. Since all other data becomes undefined when the valid signal is out of phase, only the valid signal fanout tree requires the

resettable version. This significantly reduced reset fanout, as reported by Vivado.

E. Argmx:

Our first argmax implementation was a simple comparison that involved around 30 muxes. This implementation, although simple and had a latency of one cycle, was terrible for timing and capped out frequency very early on at around 250Mhz. Most of the critical path reported was logic delay as opposed to routing

In order to remedy this we compare the popcount max over the course of 4 stages. Sink reg the input -> compare 10 -> compare 5 -> compare 3 -> compare 2 -> output. Each compare stage has its own sink registers to sink the compare outputs. Thus in total our argmax is a pipeline with an initial latency of 8.

F. Buffering

Skid Buffer: *Our Input_Layer_Unit has the AXI image interface pass through Skid_Buffer before the binarization stage so that if the first hidden layer stalls then the current beat is held in the skid register instead of being lost. I like to think of a skid buffer as a future protection plan. This also allowed us to simplify the downstream(essentially every other module in our BNN) considerably.*

Width Conversion Input FIFO: *Our width conversion fifo is another type of optimization knob. Prior to implementing this PW for the first layer was forced to be 8. Beats * groups = steady state throughput for any given layer. So if PW = 8 beats = 98. Forcing us to make groups equal to one in order to have any type of reasonable throughput for our overall architecture. Thus, forcing us to make H0(hidden layer 0) have a PN of 256. In simpler terms each neuron needed its own neuron processor. This was horrible for routing pressure.*

By implementing a width conversion fifo we were allowed to make PW of H0 any value we wanted, controlling how many neuron processors the layer had. After testing several combinations we arrived that the best for optimization was PW = 64 and PN = 32. The fifo would collect 8 binarized AXI streamers then send it to the first hidden layer for processing.

Ping-pong activation buffers between layers: *These buffers connected one layer's output activations to the next layer's input. We used two banks so that one bank can be written while the other is being read. Instead of a simple FIFO we used this because the consuming layer does not just receive the data once and discard it. It needs to be reused over several cycles.*

It is important to note that only the first hidden layers input buffer sends stall logic to the AXI upstream. This made it possible to use simpler input buffer modules for the other two layers.

Memory-boundary sink buffers: *We added these to prevent the layer controller from directly driving the memory which would create a long critical path. Our design captures the address, write enable, and data locally so that the memory then sees a signal from a closer register. We made two of these, one for the block RAM mainly used for weights and one for LUTRAM used for the thresholds.*

Output Buffer: *To solve this backpressure issue we added an output FIFO with a depth of 16.*

G. Configuration

The configuration part of this project was one of the more difficult ones to implement and even harder to optimize. In our module we have a configuration manager for the whole system and 3 configuration layer controllers(one for each hidden layer). The last hidden layer has a simplified version of this controller due to the fact it does not issue thresholds.

Configuration Manager:

The configuration manager collects 64-bit payloads from AXI and delivers the necessary data to the individual configuration layer controllers. First, the manager extracts the required header data. Our system is designed such that the manager only needs to send 3 bits of header information to the configuration controllers: 1 bit for message type and 2 bits for layer type. In hindsight, this could likely be reduced to 2 bits, since once two types are completed, only one remains. This would further optimize the system.

The remaining header information is retained within the configuration manager: 15 bits for the total number of bytes and 9 bits for the total number of neurons. Using this information, the configuration manager determines how many payloads to send to each configuration layer controller. In total, only 3 bytes are stored in the manager, significantly reducing resource usage and area. Once the necessary header fields are collected, the manager sends out a msg_valid pulse. Because msg_valid is a pulse, there is no need to reset the fields sent to the controllers.

The manager then transitions to the payload states. In these states, it collects the 64-bit AXI payload along with the associated tkeep byte mask. It then sends the data out one byte at a time to the configuration layer controllers. For example, if tkeep = 00101000, the 4th and 6th bytes are skipped while the remaining bytes are sent. Once the handshake between the manager and the configuration controller is confirmed, the byte is transmitted. Due to the pulse-like nature of this handshake, very few registers need to be reset. If the transmitted data is not perfectly aligned in phase with either system, it does not affect correctness.

Originally, the configuration manager was controlled by a large and complex state machine with over 12 states.

Prior to reaching 850 MHz, this was not a bottleneck. However, beyond 850 MHz, nearly every critical path involved either a state bit acting as a clock enable (CE) or a reset (R). To address this, we reduced the state machine to four states and pipelined much of the logic previously handled by the state bits. This increased our frequency beyond 900 MHz and reinforced an important lesson: when optimizing for high frequencies, less is more.

Configuration Layer Controller:

Each individual layer in our system has its own configuration controller. For the first two hidden layers, it is responsible for writing both weights and thresholds to their respective memories. For the final layer, it only writes the necessary weights.

The configuration layer controller begins by collecting the required header information and determining whether it will send weights or thresholds through a scheduler. This scheduler also sets up the control signals needed for the rest of the datapath to operate correctly. Due to the simplifications in the final layer, it does not need to make this decision—it always sends weights and, once completed, will only receive new data after a reset.

For weights, the controller collects enough bytes to construct a single neuron. The neuron is then built using a pipelined module labeled Build_W.sv. After construction, the controller sends out PW-length words to the corresponding weight RAM downstream. Once all neurons have been processed, the controller returns to the scheduler and waits for thresholds.

Threshold handling is significantly simpler, as it requires less logic. The controller collects 4 bytes to form a 32-bit threshold word and sends it to the appropriate threshold RAM.

This component was by far the most difficult and complex part of the system to optimize. Initially, the controller was implemented as a large 22-state FSM. Similar to the configuration manager, clock enable (CE) and reset (R) paths became critical after 800 MHz. To address this, we first divided the controller into three smaller controllers: a scheduler, a weight controller, and a threshold controller. This brought us closer to 850 MHz but was still not optimal.

We then gradually reduced the reliance on the state machine by minimizing clock enables and resets. This was achieved by gating signals with valid signals and restructuring the architecture so that misalignment with the valid pipeline would not affect correctness. Additionally, since the valid signals sent downstream (to the NP weight and threshold memories) are pulses, the data itself does not need to be perfectly aligned when written.

By continuously reducing state-dependent control and increasing structural pipelining, we were eventually able to remove the FSM entirely. This fully structural configuration layer controller has almost no clock enables or resets, allowing us to exceed 900 MHz.

Verification

A. Coverage

For all of the coverage groups defined below except cg_data_in_inter_image_gap we achieved 100 percent coverage with the defined bins using directed tests and the existing tests from provided testbench. If we had more time we would have implemented more directed tests based on the coverage plan. This is all checked in bnn_fcc_coverage_tb.sv.

Configuration bus: *To verify the config bus worked fine under different conditions we set up coverage across many cases.*

- *Both full-width and partial t-keep values occurring*
- *Beats arriving both continuously and after a gap*
- *Configuring of system in all orders (weights -> weights, weights ->thresholds, thresholds-> weights, thresholds->thresholds)*
- *Reserved header field fed with non-zero values*
- *Stall cycles accumulated in a burst from 0 to 200*
- *Stalls occurring on header and payload beats*
- *Gaps between 2 config messages from 0 to 500 cycles*
- *Single and mult-beat messages for both message types*

Data Bus:

- *Various ranges of bursts or consecutive beats without gaps from 0 to 513*
- *Gap lengths before a beat from 0 to 500*
- *Gap lengths occurring on last beat of image*
- *Pixel content with all zeros, all ones, and mixed*
- *Full and partial t-keep occurring during tests*
- *Idle gaps between images from 0 to 1000 cycles*

Output Bus:

- *Backpressure present before each output accepted*
- *Backpressure durations from 0 to 1000 cycles*
- *All classes predicted by DUT*
- *Backpressure occurring at exact output handshake*

Input Diversity:

- *Uniform images with all pixels identical*
- *Low spread images*
- *High spread images*

Scenarios:

- *Reset occurring during idle, in the middle of configuration, in the middle of inputs, and in the middle of output*

- *Reset interrupting partial config*
- *Only weights and thresholds sent before reset*
- *Reset when t-last asserted on config bus*